

Test 23-24_MAS Exam February RESIT



Test ID: 216388 (In progress)

Folder: /Top/SIS/XB_0024

Version: 2.0

Randomised: No

Last modified: Tuesday, 2 december 2025 16:45:00

Number of questions: 29

Blocks: Fixed

Display questions once: No

Tools: None

Test time: -

Cut-off: None

Maximum score: 100 pt.

Chance score: 3.11 pt. / 3%

Mark scale: -

Grading series: -

Scoring rule: No point deduction for incorrect answer

In test set with: -

Test instruction

Welcome to the exam of the course Multi-Agent Systems (XB_0024). This exam will test your knowledge and understanding of the material provided to you and presented in the lectures, the book of Blackburn, Bos and Striegnitz Learn Prolog Now! (Chapters 1 to 6, 10 and 11), and the MARBEL Programming Guide. It is not allowed to use materials such as books, scientific papers or slides during the exam. You will have 2:45 hours to complete the exam.

The following tools are permitted:

- Pen and drafting paper.

For questions about the content of the test, you can contact the teacher. Information about the review of the exam is provided via Canvas.

If you have **not** signed up for this exam, you will not receive a result. Through VUnet you can object to the fact that you can no longer sign up after the expiry of the registration deadline (and the fact that you will not receive a result for this exam). Submit your appeal online within one week after the exam. More information can be found at www.vu.nl/intekenen.

Good luck!

Welkom bij de toets van het vak Multi-Agent Systems (XB_0024). Deze toets test je kennis en begrip van de stof als gepresenteerd in de colleges, het boek van Blackburn, Bos en Striegnitz Learn Prolog Now! (Hoofdstukken 1 tot 6, 10 en 11), en de MARBEL Programming Guide. Het is niet toegestaan om hulpmiddelen zoals boeken, publicaties en slides te gebruiken tijdens de toets. Je hebt 2:45 uur om de toets te maken.

Bij deze toets mag je gebruik maken van de volgende hulpmiddelen:

- Pen en kladpapier.

Voor vragen over de inhoud van de toets kun je bij de docent terecht. Informatie over de toetsinzage wordt via Canvas verstrekt.

Als je **niet** hebt ingetekend voor dit tentamen, ontvang je geen resultaat. Via VUnet kun je bezwaar maken tegen het feit dat je niet meer kunt intekenen na het verstrijken van de intekendeadline (en je dus geen resultaat krijgt voor dit tentamen). Dien je bezwaarschrift online in binnen één week na het tentamen. Meer informatie vind je op www.vu.nl/intekenen.

Veel succes!

Prolog queries

Question order: Fixed

The following questions each give a Prolog query, after which you are asked to fill in the outcome that Prolog will give based on the query.

Question 1 – 23-24_prolog_query_1 RESIT – 415492.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 19:12

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

?- time = money.

false.

false

Question 2 – 23-24_prolog_query_2 RESIT – 415489.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 19:15

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

?- f(a,g(b))=f(a,g(X)).

X = b.

X = b

X=b.

X= b

X= b.

X =b

X =b.

Question 3 – 23-24_prolog_query_3 RESIT – 415490.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 19:16

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

?- f(g(X,y,Z)) = f(g(Z,x,Y)).

false.

false

Question 4 – 23-24_prolog_query_4 RESIT – 415491.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 19:20

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

?- X*2 = 7*Y.

X = 7, Y = 2.

X = 7, Y = 2

X=7, Y=2.

X=7, Y=2

X=7,Y=2.

X=7,Y=2

X = 7,Y = 2.

X = 7,Y = 2

X= 7, Y = 2.

X= 7, Y = 2

X= 7, Y= 2.

X= 7, Y= 2

Y = 2, X = 7.

Y = 2, X = 7

Question 5 – 23-24_prolog_query_5 RESIT – 415488.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 19:21

Attributes: Taxonomie/Taxonomy 00 Onbekend/Unknown

?- X is 5 x 5.

error.

error

false

false.

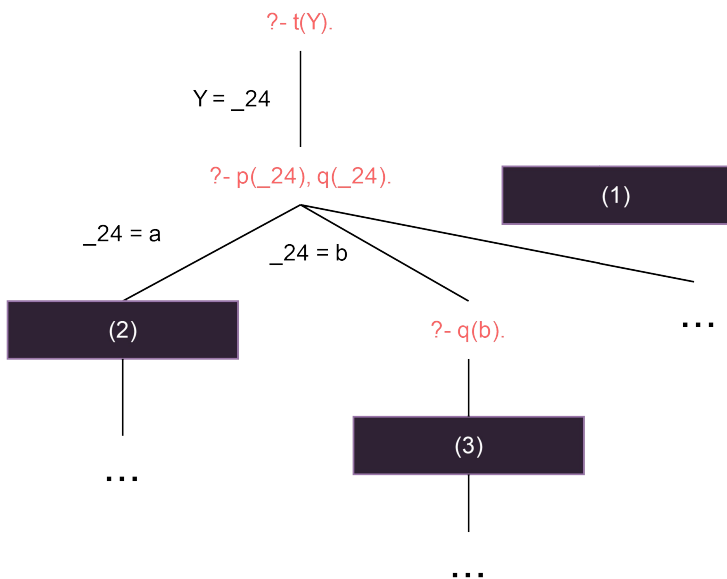
Prolog search

Question order: Fixed

The questions in this block will be based on the following knowledge base:

p(a).
p(b).
p(c).
q(X) :- v(X), w(X).
t(X) :- p(X), q(X).
v(a).
v(c).
w(b).
w(c).

Part of the search tree for the query ?- t(Y) is given:



Question 6 – 23–24_prolog_search_1 RESIT – 415493.2.0

Question type: Multiple choice

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Random

Partial scoring: No

Maximum score: 2

Chance score: 0.25 pt. / 13%

Status:

Last modified: 05/02/2024 18:45

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Which branch leads to a complete solution?

- A** Left branch
- B** Middle branch
- C** Right branch
- D** None of the branches
- E** All of the branches
- F** The left and the middle branch
- G** The left and the right branch
- H** The middle and the right branch

Question 7 – 23–24_prolog_search_2 RESIT – 415494.2.0

Question type: Multiple choice

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Random

Partial scoring: No

Maximum score: 2

Chance score: 0.33 pt. / 17%

Status:

Last modified: 05/02/2024 18:54

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

What happens when a branch in a search tree cannot be completed?

- A** Through *backwards chaining* the closest possible previous node is explored for an alternative route.
- B** Through *backtracking* the closest possible previous node is explored for an alternative route.
- C** Through *linear traversal* the closest possible previous node is explored for an alternative route.
- D** Through *backwards chaining* the topmost node that has unexplored options is explored further for an alternative route.
- E** Through *backtracking* the topmost node that has unexplored options is explored further for an alternative route.
- F** Through *linear traversal* the topmost node that has unexplored options is explored further for an alternative route.

Question 8 – 23-24_prolog_search_3 RESIT – 415495.3.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 18:56

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Fill in box (1) in the search tree displayed above.

_24 = c

_24 = c.

_24=c

_24=c.

_24= c

_24= c.

_24 =c

_24 =c.

Question 9 – 23–24_prolog_search_4 RESIT – 415496.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 5

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 18:58

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Fill in box (2) in the search tree displayed above.

?- q(a).

?- q(a)

q(a).

q(a)

?-q(a).

?-q(a)

?- q(a).

?- q(a).

?- q(a)

?- q(a)

?- q(a).

?- q(a)

Question 10 – 23-24_prolog_search_5 RESIT – 415497.2.0

Question type: Fill in

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: No

Maximum score: 5

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 19:00

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Fill in box (3) in the search tree displayed above.

?- v(b), w(b).

?- v(b), w(b)

v(b), w(b).

v(b), w(b)

?- v(b),w(b).

?- v(b),w(b)

v(b),w(b).

v(b),w(b)

?-v(b), w(b).

?-v(b), w(b)

?-v(b),w(b).

?-v(b),w(b)

Prolog concepts

Question order: Fixed

The following questions address your knowledge of Prolog concepts. Select none, one, or more answers from a list of answers options.

Question 11 – 23-24_prolog_concepts_1 RESIT – 415498.3.0

Question type: Multiple response

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Random

Partial scoring: Yes

Maximum score: 2

Chance score: 0.59 pt. / 30%

Status:

Last modified: 19/02/2024 10:37

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

If t1 and t2 are compound terms, they match if (Select **all** the correct options)

- ☐ A All the terms in t1 match with all the terms in t2.
- ☒ B The terms in t1 match pairwise with the terms in t2.
- ☐ C Each variable in t1 matches with a variable in t2.
- ☒ D t1 and t2 have the same outermost functor.
- ☐ E the outermost functor of t1 and t2 are different.
- ☐ F Each constant in t1 matches with a constant in t2.

Question 12 – 23-24_prolog_concepts_2 RESIT – 415499.2.0

Question type: Multiple response

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Random

Partial scoring: Yes

Maximum score: 2

Chance score: 0.60 pt. / 30%

Status:

Last modified: 05/02/2024 18:42

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

A predicate *pred* is defined recursively if there is at least one rule in the definition of *pred*, (select **all** the correct options).

- ☒ A in which *pred* itself is reachable.
- ☐ B in which a list is separated into a head and a tail.
- ☐ C in which the iterative variable is unified.
- ☐ D in which backtracking is applied.

Programming in Prolog

Question order: Fixed

The following two questions will be programming exercises. You will have to create a Prolog program, using correct Prolog syntax, defining one or more predicates to answer each question.

Question 13 – 23-24_prolog_programming_1 RESIT – 415500.1.3

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 10

Chance score: 0.00 pt. / 0%

Status:

Last modified: 19/02/2024 10:38

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 500

Word count: No

Define a predicate: *replace*(O, R, List1, List2) that is true if List2 is the same as List1, but with all elements that match with O replaced with R. For example, *replace*(a, q, [a,b,c,a,b,c], [q,b,c,q,b,c]). succeeds.

You are **required** to use *recursion*.

You are **not** allowed to use any built-in predicates.

Grading instruction

Answer:

replace(_, _, [], []).

replace(O, R, [O|T], [R|T2]) :- *replace*(O, R, T, T2).

replace(O, R, [H|T], [H|T2]) :- H \= O, *replace*(O, R, T, T2).

10 points

Criterion 1 (Number of points: 2)

Define the input and output lists in its separated form (i.e. [H | T]) in the head of the rule and recurse by calling *replace* in the body of the rule with the same O, and R and the tails of both the input and output list.

For example, *replace*(O, R, [_|T], [_|T2]) :- *replace*(O, R, T, T2).

Criterion 2 (Number of points: 3)

Base clause stops recursion at the appropriate moment. For example: *replace*(_, _, [], []).

-2 if both lists are not empty.

-1 if elements are not anonymous.

Criterion 3 (Number of points: 3)

Define two rules:

1. The head of the input list is equal to element O (i.e. the one that needs to be replaced with R): replace the element (and recursively continue).
2. The head of the input list is not equal to element O: don't replace.

Criterion 2 (Number of points: 2)

Do the replacing directly in the head of the rule, e.g. *replace*(O, R, [O|T], [R|T2]).

Question 14 – 23-24_prolog_programming_2 RESIT – 415501.1.3

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 10

Chance score: 0.00 pt. / 0%

Status:

Last modified: 19/02/2024 10:39

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 500

Word count: No

Define a predicate: *mirror(List1, List2)* that is true if List2 is a mirrored version of List1. For example, *mirror([1,2,3], [3,2,1])*. succeeds.

You are **required** to use an *accumulator* in your solution.

You are **not** allowed to use any built-in predicates.

Grading instruction

Answer:

mirror(L, M) :- mirror_acc(L, [], M).

mirror_acc([], R, R).

mirror_acc([X|Xs], Ys, R) :-

mirror_acc(Xs, [X|Ys], R).

10 points

Criterion 1 (Number of points: 2)

Use of accumulator with arity 3. For example *mirror_acc(List1, Accumulator, List2)*.

Criterion 2 (Number of points: 2)

Call accumulator from main wrapper predicate, with an empty list as initial accumulator "value".

Criterion 3 (Number of points: 2)

Define the base clause of the accumulator to "return the accumulated" after stepping through the list once (i.e. the input list is empty). For example, *mirror_acc([], R, R)*.

Criterion 4 (Number of points: 4)

Utilize the core purpose of the accumulator to unidirectional step through the stack to "collect" the elements of the list in reverse order. For example,

mirror_acc([X|Xs], Ys, R) :-

mirror_acc(Xs, [X|Ys], R).

Agents and MARBEL: Basics

Question order: Fixed

This part tests your basic knowledge of agents and MARBEL programming concepts.

Question 15 – 23-24_MARBEL_Basics_1a1 RESIT – 415473.1.2

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 15:47

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 20

Word count: Yes

Environments that agents can connect to can be classified according to different properties.

Explain what it means for an environment to be a *deterministic/stochastic* environment.

Grading instruction

Criterion 1 (Number of points: 1)

The environment (does not) only changes...

Criterion 2 (Number of points: 1)

...when an agent performs an action.

Question 16 – 23-24_MARBEL_Basics_1a2 RESIT – 415474.1.1

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 15:48

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 15

Word count: Yes

Apart from the deterministic/stochastic property, mention two further properties that environments can have which are relevant for designing agents.

Grading instruction

Criterion 1 (Number of points: 1)

One of:

- fully/partially observable
- static/dynamic
- discrete/continuous
- single/multi-agent

Criterion 2 (Number of points: 1)

Another one of:

- fully/partially observable
- static/dynamic
- discrete/continuous
- single/multi-agent

Question 17 – 23-24_MARBEL_Basics_1b RESIT – 415475.1.1

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 4

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 17:25

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 30

Word count: Yes

Why does an agent need to receive percepts from an environment if that environment is dynamic?

Grading instruction

Criterion 1 (Number of points: 2)

if dynamic, agent needs to "see" what has changed by the environment...

Criterion 2 (Number of points: 2)

... because things change that the agent did not change itself.

Question 18 – 23-24_MARBEL_Basics_1c1 RESIT – 415476.1.1

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 05/02/2024 17:27

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 30

Word count: Yes

What is a so-called *send always* percept?

Grading instruction

Criterion 1 (Number of points: 1)

A percept that is sent always...

Criterion 2 (Number of points: 1)

... when the environment feature is present.

Question 19 – 23-24_MARBEL_Basics_1c3 RESIT – 415478.2.0

Question type: Multiple choice

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Random

Partial scoring: No

Maximum score: 2

Chance score: 0.67 pt. / 33%

Status:

Last modified: 05/02/2024 17:30

Attributes: *Taxonomie/Taxonomy* 01 Kennis/Knowledge

Which channel handler can be best used to handle *send always* percepts?

A The *add* handler.

B The *replace* handler.

C The *update* handler.

Question 20 – 23-24_MARBEL_Basics_1c4 RESIT – 415479.1.2

Question type: Multiple choice

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Random

Partial scoring: No

Maximum score: 4

Chance score: 0.67 pt. / 17%

Status:

Last modified: 12/02/2024 11:00

Attributes: *Taxonomie/Taxonomy 00 Onbekend/Unknown*

Where should code for handling a percept be placed (in what type of file, and in which part of that file)?

- A** In a `mas2g` file, as part of the dynamic predicate declaration.
- B** In a Prolog file, as part of the dynamic predicate declaration.
- C** In a `mod2g` file, as part of the use clauses.
- D** In a `mod2g` file, as part of the program section.
- E** In a Prolog file, as part of the launch policy section.
- F** In a `mas2g` file, as part of the agent definition.

Question order: Fixed

This assignment is about a (toy) pick-and-place sorting robot that is deployed in a warehouse to sort blue and red cans into the corresponding blue or red box. Cans are labelled with numbers 1, 2, etc. which allows the robot to distinguish them from each other. The predicate *color/2* is used to indicate the color of a can. *color(1,red)* and *color(2,blue)*, for example, indicate that can 1 is red and can 2 is blue. Cans are either in a box or on a table. The robot is able to pick up cans and put them in a box or on a table. The predicates *in/2* and *on/2* are used to represent where a can is. For example, *in(1,blue)* means that can 1 is in the blue box and *on(2,table)* that can 2 is on the table. Note that a red can can be in the blue box. The can environment is fully observable. The robot's gripper, however, is not that reliable and sometimes drops a can when it is holding it on the table. Carefully analyse the MARBEL agent system in the Figure below.

Also make sure to follow the instructions given below.

Instructions: When you add code in the program templates *or* are asked to explain how the program works, **always** clearly specify the position in the code where you want to add something or that you are referring to in your explanation of the code. If you want to *modify* a line, for example, number 11, in a file named 'file.ext', and replace it with code '...', use the following notation:

file.ext 11 ...

If you want to place *new* code '...' in some file between two line numbers, for example, 23 and 24, use the following notation:

file.ext 23.1 ...

file.ext 23.2 ...

file.ext 23.3 ...

```

----- sortRobot.maszg -----
1 use "sensors-and-actuators.jar".
2
3 define sortRobot {
4   use sortInit for init.
5   use sortAction for decisions.
6
7   ...
8   ...
9   ...
10 }
11
12 launchpolicy {
13   when * launch sortRobot.
14 }
-----
----- sort.pl -----
1 :- dynamic color/2, in/2, on/2.
2
3 % The box Box is empty (no cans can be found in it).
4 empty(Box) :- findall(C, in(C, Box), []).
5 % The robot's gripper is holding can Nr.
6 holding(Nr) :- color(Nr, _), ... .
-----
----- sortInit.modzg -----
1 use sort.
2
3 module sortInit {
4   if true then insert(start).
5   ...
6 }
-----
----- sortAction.modzg -----
1 use sort.
2
3 module sortAction {
4   % First empty the boxes...
5   if start then emptyBoxes.
6
7   % If the robot is holding a can, put it in the right box.
8   if holding(Nr), color(Nr, C) then put(Nr, C).
9   % If the gripper is empty and a can needs to be moved, pick it up.
10  if gripperEmpty, ... then pickup(Nr).
11 }
-----
----- emptyBoxes.modzg -----
1 use sort.
2
3 exit=never.
4
5 module emptyBoxes {
6   if in(Nr, Box) then pickup(Nr).
7   if holding(Nr) then put(Nr, table).
8   if empty(red), empty(blue) then exit-module.
9 }

```

Figure 1: The sortRobot agent.

Question 21 – 23-24_MARBEL_Programming_2a RESIT – 415480.2.2

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 5

Chance score: 0.00 pt. / 0%

Status:

Last modified: 27/02/2024 09:39

Attributes: *Taxonomie/Taxonomy* 03 *Toepassen/Apply*

Word count: 50

Word count: Yes

The robot's gripper is initially empty, i.e. it is not holding anything. The predicate *gripperEmpty* is used to represent that fact. Add an action rule to the code in the most appropriate place (indicate in which file) to make sure that the agent knows its gripper is empty, i.e. *gripperEmpty* is in its knowledge base. What else needs to be added to the program to make it work without problems (again, indicate also where)?

Grading instruction

Criterion 1 (Number of points: 3)

`sortInit.mod2g 4 if true then insert(gripperEmpty).`

The predicate needs to be declared.

1 point for the right location, 1 point for the rule, and 1 point for the declaration. Deduct a point for any mistake made.

Criterion 2 (Number of points: 2)

The predicate needs to be declared. In `sort.pl` 1 :- `dynamic color/2, in/2, on/2, gripperEmpty/0.`

1 point for the right location and 1 for the declaration. Deduct a point for any mistake made.

It is also fine to use `forall-do` instead of `if-then`. If some mistake(s) in rule but more or less still OK, deduct 1pt. 1pt for putting rules in correct place, i.e. the init module. 2pt for each rule.

Question 22 – 23-24_MARBEL_Programming_2aa RESIT – 415502.1.1

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 3

Chance score: 0.00 pt. / 0%

Status:

Last modified: 06/02/2024 10:49

Attributes: *Taxonomie/Taxonomy 03 Toepassen/Apply*

Word count: 50

Word count: Yes

Complete the incomplete Prolog rule. For defining the `holding/1` predicate, use the fact that the robot is holding a can when it is not on the table or in a box. To avoid issues with floundering in this case, make sure that there are no ungrounded negated facts used in the rule when it is called.

Grading instruction

Criterion 1 (Number of points: 3)

```
sort.pl 6 holding(Nr) :- not(on(Nr, table)), not(in(Nr, red)), not(in(Nr, blue)).
```

1 point for right location. 1 point for query. 1 point for making sure that floundering does not happen.

Question 23 – 23-24_MARBEL_Programming_2b RESIT – 415481.2.0

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 2

Chance score: 0.00 pt. / 0%

Status:

Last modified: 06/02/2024 10:51

Attributes: *Taxonomie/Taxonomy 00 Onbekend/Unknown*

Word count: 50

Word count: Yes

Assume that `color/2` is a *send once percept* informing the agent of the colors of all the cans. Add a channel for handling this percept with the appropriate handler to the code and indicate where you would add this.

Grading instruction

Criterion 1 (Number of points: 2)

```
sortRobot.mas2g, line 7 add color/2.
```

Correct place `mas2g` (1p) and correct handler `'add'` (1p). Deduct a point for any mistake.

Question 24 – 23-24_MARBEL_Programming_2c RESIT – 415482.2.2

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 4

Chance score: 0.00 pt. / 0%

Status:

Last modified: 27/02/2024 21:09

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 50

Word count: Yes

The `in/2` and `on/2` are *send always percepts* informing the agent about whether a can is in a box or on the table. Add appropriate handlers for these two percepts and indicate where this should be put in the code.

Grading instruction

Criterion 1 (Number of points: 2)

`sortRobot.mas2g` line 8 replace `in/2`.

Criterion 2 (Number of points: 2)

`sortRobot.mas2g` line 9 replace `on/2`.

Question 25 – 23-24_MARBEL_Programming_2d RESIT – 415483.1.2

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 3

Chance score: 0.00 pt. / 0%

Status:

Last modified: 12/02/2024 11:10

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 50

Word count: Yes

Complete the code in `sortAction.mod2g` line 10 and replace the `...`, to ensure the gripper is functioning correctly. Do not introduce any new predicates, but use one or more of the predicates listed in the listing to specify the query of the action rule. For this question, please *only specify the query*, as the location is already given.

Grading instruction

Criterion 1 (Number of points: 3)

Replace `...` with `on(Nr, table)`

Any issue deduct one point.

Question 26 – 23-24_MARBEL_Programming_2e RESIT – 415484.1.1

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 4

Chance score: 0.00 pt. / 0%

Status:

Last modified: 06/02/2024 10:56

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 50

Word count: Yes

Explain what the `exit=never` option that is used in the `emptyBoxes` module means.

Grading instruction

Criterion 1 (Number of points: 4)

It means that once entered the module is never exited and the rules in the module will be evaluated over and over again (without using an explicit `exit-module` action).

3 points for the first part, and 1 point for the fact that the rules are evaluated over and over again.

Question 27 – 23-24_MARBEL_Programming_2f RESIT – 415485.3.0

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 4

Chance score: 0.00 pt. / 0%

Status:

Last modified: 06/02/2024 11:06

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 500

Word count: No

The current program will keep trying to make the boxes empty by calling the `emptyBoxes` module over and over again in line 5 of `sortAction.mod2g`. Fix this problem. *Hint:* Think about what the agent needs to do before exiting the `emptyBoxes` module. Indicate which code you changed how and where.

Grading instruction

Criterion 1 (Number of points: 4)

`emptyBoxes.mod2g` line 8 `if empty(red), empty(blue) then delete(start) + exit-module.`

1pt for correct location. 1pt for using the `+` operator. 2pts for the delete action. Deduct 1pt if any mistake is made.

Question 28 – 23-24_MARBEL_Programming_2g RESIT – 415486.3.0

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 4

Chance score: 0.00 pt. / 0%

Status:

Last modified: 06/02/2024 11:05

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 500

Word count: No

Why can't the percept $on/2$ be a send once percept? Explain why the agent also needs to receive the $in/2$ percepts and these are not redundant (why is it not sufficient to *only* receive the $on/2$ percepts?).

Grading instruction

Criterion 1 (Number of points: 4)

The $on/2$ percept needs to be a send always percept because the robot can drop cans which end up on the table.

The $in/2$ percept is needed to be able to conclude that the robot is holding a can.

2 pts for each explanation.

Question 29 – 23-24_MARBEL_Programming_2h RESIT – 415579.2.0

Question type: Open-ended

Pre-test item: No

Folder: /Top/SIS/XB_0024

Folder description: Multi-Agent Systems

Answer option order: Fixed

Partial scoring: Yes

Maximum score: 5

Chance score: 0.00 pt. / 0%

Status:

Last modified: 12/02/2024 11:06

Attributes: *Taxonomie/Taxonomy* 00 Onbekend/Unknown

Word count: 500

Word count: No

Even though the robot can drop cans, eventually, when it picks up a can, it will put the can in a box because the probability of dropping a can is only 10%. Given this fact, explain why this agent program will succeed in putting all the cans in the right boxes (assuming the problem with the continuous execution of the `emptyBoxes` module has been fixed).

Grading instruction

Criterion 1 (Number of points: 1)

First, note that the number of cans does not change (it remains fixed after starting the agent).

Criterion 2 (Number of points: 1)

Because in the `sortInit.mod2g` line 4 the start fact is inserted, the agent then will first empty the boxes by applying the action rule on line 5 in `sortAction.mod2g`.

Criterion 3 (Number of points: 1)

After doing that, all cans will be on the table (see line 7 of `emptyBoxes.mod2g`).

Criterion 4 (Number of points: 2)

Thereafter, only the action rules on line 8 and 10 in `sortAction.mod2g` will be executed, which move the cans into the right boxes (1pt for mentioning each line).

Test matrix

No matrix defined